

Guia de Arquitetura de Software + IA

Antônio Lucas de Lima Gueiros Pires

Fundamentos

Design Patterns (GoF)

- **Singleton (Criacional)**

Um Singleton é um padrão de projeto que garante que apenas uma instância (objeto) de uma classe exista em toda a aplicação — e que todos os lugares do código usem essa mesma instância.

- Características:
 - Não é muito usado atualmente (pode complicar testes).
 - Se assemelha a uma variável global, mas com controle.
- Usos comuns:
 - Conexão única com banco de dados (DB).
 - Acesso centralizado ao sistema de arquivos.
 - Sistema de logs.
- Por que evitar várias instâncias?
 - Gastam memória à toa.
 - Podem gerar inconsistências (cada instância teria dados diferentes).

- **Factory (Criacional)**

O Factory é um padrão que centraliza a criação de objetos. Em vez de você usar `new` direto no código, você pede para a “fábrica” criar e devolver o objeto certo.

- Benefício: desacopla a criação do uso.
 - Exemplo: Em um sistema de IA, uma Factory pode decidir se cria um modelo GPT, um BERT ou um LSTM, sem que o código principal precise saber os detalhes.
-

- **Strategy (Comportamental)**

O Strategy permite trocar o comportamento de um algoritmo em tempo de execução, sem mudar o código que o usa. Você define uma “família” de estratégias, e escolhe qual aplicar.

- Exemplo:

- Um sistema de recomendação de IA pode usar diferentes estratégias: colaborativa, baseada em conteúdo ou híbrida.
- O código que usa a recomendação continua o mesmo, apenas a estratégia muda.

- **Observer (Comportamental)**

O Observer serve para notificar vários objetos quando algo acontece. É o famoso padrão “publicador/assinante” (publisher/subscriber).

- Exemplo:

- Em um app de IA que monitora sensores, quando um sensor muda de valor, todos os módulos interessados (ex.: log, alerta, IA de previsão) são notificados automaticamente.

- **Adapter (Estrutural)**

O Adapter é usado para conectar interfaces diferentes, funcionando como um “tradutor”. Permite que duas classes que não foram feitas para trabalhar juntas consigam se comunicar.

- Exemplo:

- Você tem um modelo de IA que gera saída em JSON, mas sua aplicação só entende XML.
- Um Adapter pode converter automaticamente de JSON → XML, sem mudar o modelo nem a aplicação.

★ **Exemplo prático implementado em Python:**

🔗 <https://github.com/AntonioPROLucas/DesignPatterns-Python>

12 Fatores

Os doze fatores necessitam ser satisfeitos para desenvolver uma aplicação web escalável independente da linguagem de programação ou framework! Listado pela primeira vez em um site criado por Adam Wiggins (co-fundador da Heroku).

1. Codebase (Base de Código Única)

- Uma aplicação deve ter um único repositório de código por serviço.
- Você pode ter múltiplos deploys (produção, teste, dev), mas todos vêm do mesmo código.

Exemplo:

- GitHub/GitLab com branches para dev/test/prod.

2. Dependências

- Todas as dependências (bibliotecas externas) devem estar declaradas em um arquivo de manifesto.
- Nunca instale algo “na mão” na máquina.
- Assim, qualquer dev ou servidor baixa as mesmas dependências com um único comando.

Exemplo:

- Node: package.json + npm install • Python: requirements.txt + pip install -r requirements.txt

3. Configurações

- Configurações variam de ambiente para ambiente (senhas, tokens, URLs).
- Elas não devem estar no código.
- Devem ficar em variáveis de ambiente (.env).

Exemplo: DATABASE_URL=postgres://user:senha@localhost:5432/db

4. Serviços de Apoio (Backing Services)

- Qualquer serviço externo (banco de dados, fila, cache, API externa) deve ser tratado como recurso anexado.
- A aplicação não deve “sentir diferença” entre usar um serviço local ou um externo.

Exemplo:

- Trocar o Redis local por um Redis na nuvem sem alterar o código.

5. Build, Release, Run

- O ciclo de vida deve estar bem separado:
 - Build → compila o código e resolve dependências.
 - Release → junta o build + configs para rodar.
 - Run → executa a aplicação.

💡 Assim, se algo quebra, você sabe se foi na build, configuração ou execução.

6. Processos

- A aplicação deve ser sem estado: nada fica guardado na memória do servidor entre requisições.
- Se precisar de estado (ex.: sessão de usuário), use serviços externos (Redis, banco).

💡 Isso permite escalar horizontalmente (ter várias instâncias rodando ao mesmo tempo).

7. Port Binding (Vinculação por Porta)

- A aplicação deve ser autossuficiente, expondo serviços via uma porta.
- Nada de depender de um servidor externo (como Apache/Nginx embutido).

Exemplo: Um app Node expõe a API direto na porta 3000.

8. Concorrência

- A aplicação deve escalar rodando múltiplos processos da mesma base.
- Ao invés de “ficar gigante” em um servidor só, ela roda em vários menores.

Exemplo: Ter 10 instâncias da API rodando, dividindo a carga.

9. Descartabilidade (Disposability)

- Processos devem ser rápidos de iniciar e fáceis de desligar.
- Isso permite deploys rápidos e escalabilidade elástica.

Exemplo:

- Um app que sobe em segundos e pode ser derrubado sem travar nada.
-

10. Dev/Prod Parity (Paridade entre Ambientes)

- O ambiente de desenvolvimento, teste e produção devem ser o mais parecido possível.
- Assim você evita o famoso: “Na minha máquina funciona”.

Exemplo: Usar PostgreSQL no dev e PostgreSQL no prod (e não SQLite no dev e PostgreSQL no prod).

11. Logs

- A aplicação deve gerar logs simples em texto e jogar no stdout (saída padrão).
- Quem deve guardar, processar e analisar os logs é o sistema de execução (não a aplicação).

Exemplo:

- O app só imprime: User 123 logged in.
 - O sistema (Docker, Heroku, AWS) coleta e armazena.
-

12. Administração

- Tarefas administrativas (migração de banco, scripts de manutenção) devem ser executadas como processos pontuais, usando o mesmo código e ambiente da aplicação.

Exemplo:

- Rodar `python manage.py migrate` no mesmo container/app.
-

Arquitetura de Sistemas de Alto Nível (A Camada de Maturidade 1.)

- **Arquitetura Hexagonal:** Busca desacoplar a aplicação de seu ambiente. O código é separado em camadas isoladas do mundo externo, que se comunica através de **Ports e Adapters**.
- **Ports(portas):** Interfaces genéricas que protegem o que entra na sua camada de domínio, trocando apenas o adaptador (**adapter**), sempre que quiser acrescentar uma estrutura externa.
 - **Driving Ports (Primary):** Interfaces que o mundo externo utiliza para interagir com a aplicação (ex: **IUserService**).
 - **Driven Ports (Secondary):** Interfaces que a aplicação utiliza para solicitar dados ou ações externas (ex: **IUserRepository**).

- Exemplo:

```
// Port (Interface) - Driven/Secondary
public interface IUserRepository
{
    void Save(User user);
    User GetById(Guid id);
}
```

- **Adapters:** É o que “traduz” o mundo externo para o formato desejado pelas portas.
 - **Driving Adapters:** Traduzem requisições externas para as chamadas do núcleo (ex: Controllers, CLIs, RabbitMQ Consumers).
 - **Driven Adapters:** Implementam as interfaces de saída para interagir com tecnologias específicas (ex: SQL Repository, API Client, Cache).

```
// Adapter (Implementation)
public class SqlUserRepository : IUserRepository
{
    private readonly MyDbContext _context;

    public SqlUserRepository(MyDbContext context)
    {
        _context = context;
    }

    public void Save(User user)
    {
        // Lógica específica de Entity Framework/SQL
        _context.Users.Add(user);
        _context.SaveChanges();
    }

    public User GetById(Guid id) => _context.Users.Find(id);
}
```

Protocolos de Comunicação

- **HTTP / HTTPS**

O Protocolo de Transferência de Hipertexto (Hypertext Transfer Protocol | HTTP) é o protocolo básico para acessar páginas Web, transferindo um Hipertexto do servidor para o computador cliente. HTTPS ou Protocolo de Transferência de Hipertexto Seguro (Hypertext Transfer Protocol Safe), transfere o Hipertexto criptografado e o decripta quando já está no computador cliente!

- Ambos estão na 4ª camada (aplicação)
- HTTPS = HTTP + SSL ou HTTP + TLS (mais seguro e usado) WebSockets

- **WebSockets**

Solução para aplicações que precisam requisitar o servidor de forma contínua (chat, jogos online, etc), o WebSocket cria uma conexão contínua entre navegador e servidor!

- **REST vs gRPC**

RPC (Remote Procedure Call) é um padrão de comunicação com o modelo Cliente-Servidor que solicita o serviço de um programa em outro computador em uma rede!

Em um RPC tradicional, quando um cliente chama por um procedimento, ele fica suspenso até o servidor retornar um resultado! Enquanto no modelo assíncrono (com 2 RPCs), o cliente só tem que aguardar até a chamada de procedimento remoto ser aceita, após o resultado da chamada, ocorre uma chamada do cliente com One-Way RPC.

-**gRPC**: Solução do **Google** com implementações RPCs em larga escala, usando o modelo HTTP/2 na camada de transporte!

-**gRPC x REST**: REST (arquitetura de sistemas) usa HTTP/1 e JSON(texto). gRPC(protocolo de comunicação), usa o HTTP/2 e Protobuf(binário) para codificação do pacote mais rápida e performática.

Programação Orientada a Eventos – Mensageria

As arquiteturas habituais trabalham com requisições, podendo ser microsserviços ou monólitos. A desvantagem é que um único erro pode prejudicar o fluxo do serviço.

Arquitetura orientada a eventos centraliza a aplicação em um único serviço, onde após receber algum pedido do cliente o microsserviço envia um “evento” para esse serviço centralizador(**mensageria**) que fica responsável por saber quando ocorre os eventos aos outros microsserviços.

- **Vantagens:** Maior escalabilidade, mais instancias, maior responsividade.

- **Ferramentas:** RabbitMQ, Apache, Kafka, AWS Amazon EventBridge, Redis Pub/Sub e Redis Streams.

Como usar a IA nisso?

- **Sem IA!**

Request Service → Event Bus → Response Service

- **Com IA!** (Aqui criamos um novo serviço independente para IA (**AI Service**))

Request Service → Event Bus → AI Service

↓ decisão inteligente

↓ novo evento no Bus

Criando Agentes de IA

Recapitulando

Oque são IAs? Como surgiram?

A IA evoluiu ao nível em que vemos hoje após o advento das **Big Datas**, que nada mais são do que a quantidade massiva de dados que disponibilizamos ao usarmos os serviços da internet, mas o conceito de IA vem de muito antes.

Em 1950 **Alan Turing** criou um teste para definir o quão bem um computador simulava o comportamento humano em uma comunicação de texto, neste caso ainda não existia uma IA, mas o computador tinha uma habilidade preditiva, Turing percebeu um padrão de conversas, observando diálogos entre pessoas, e separou um conjunto de conversas mais frequentes que a máquina auto completava, e no final a pessoa devia dizer se estava conversando com uma pessoa ou uma máquina.

Em 1956 ocorreu o evento **Dartmouth Workshop** onde **John McCarthy** afirmou que para uma máquina imitar o comportamento humano, ela deveria ser dotada de uma inteligência artificial, surgindo daí o termo.

John McCarthy também criou junto ao **Marvin Misky** (um dos fundadores do Laboratório de Inteligência Artificial do **MIT**) o **Logic Theorist**, o primeiro programa de IA capaz de provar os teoremas matemáticos, que mostrou ao mundo o potencial da IA para resolver problemas complexos e os tornando referencias para ideias e técnicas para montar modelos de IA.

Primeiro chatbot, Eliza!

Em 1966, **Joseph Weizenbaum** criou **Eliza**, conhecida como o primeiro chatbot. **Eliza** foi o primeira IA a fazer processamento de linguagem natural.

A IA te respondia trabalhando com um banco de perguntas, prevendo a resposta mais provável a determinadas palavras-chave, algo parecido com o corretor automático tentando “prever” o que você quer escrever.

A IA moderna, trabalha com o banco de dados, aprendendo relações entre palavras e conceitos.

O que é um Arquiteto de Agentes de IA?

Um arquiteto de IA é a grande promessa para o futuro. Ele basicamente tem a função de criar uma Inteligência Artificial que tem o papel de executar tarefas específicas.

Redes Neurais Artificiais: Sistemas computacionais que se inspiram nas redes neurais biológicas, criando relação e associação entre dados (exemplo: foto e gato). As redes neurais artificiais são a base do Deep Learning, sendo dividida em **Nós, Layers, Pesos e Treinamento**. No contexto de agentes de IA, as redes neurais são a tecnologia invisível que faz a **arquitetura de Software** e o **RAG** funcionar.

RAG: O **RAG (Retrieval Argument Generation)** é fundamental para criar agentes precisos, que liga modelos **LLM** (como **Gemini** e **Chat GPT**) a uma base de dados externa (uma documentação sua, por exemplo) e se baseia estritamente no banco vetorial que é criado em cima dos dados obtidos através da documentação.

Chunks: Usado no contexto de **RAG**, são pequenos pedaços da base de dados externa, que é “mastigada” para a compreensão total da IA que estiver treinando. O tamanho de cada chunk (**chunk_size**) podem ser definido pelo próprio arquiteto além da sobreposição deles a partir do “**chunk overlap**” que impede que o conteúdo de cada chunk esteja cortado ou sem contexto.

Embeddings: Ou vetorização é a transformação da requisição do cliente em vetores e a aproximação dela com os **chunks** para decidir a resposta mais apropriada.

Webhook: No processo de comunicação com agentes de IA, o Webhook seria a ‘companhia’, eles basicamente notificam os agentes a partir de determinado evento (exemplo: “um cliente manda uma mensagem para o Whatsapp da sua empresa”).

Google SDK: Software Development Kit (SDK) é justamente o que seu nome dá a entender, são ferramentas já prontas, criadas pelo Google permitindo que você se conecte com seu sistema, com códigos e funções já prontas, simplificando a configuração de agentes de IA.

Exemplo:

```
from langchain_google_genai import ChatGoogleGenerativeAI # Integração do Gemini
```

O Orquestrador: Ele é quem dita como IA vai trabalhar, quais dados e ferramentas ela vai usar e como ela vai treinar. As principais ferramentas para isso são o LangChain e o Agno.

Exemplos:

```
from agno.tools.googlecalendar import GoogleCalendarTools
from agno.models.google import Gemini
```

N8N: Se o Weebhook é a 'companhia', o N8N seria o atendente que abre a porta e te encaminha para o anfitrião, sendo a ponte entre a requisição (o evento) e o Agente, o Orquestrador e qualquer ferramenta que faça parte do projeto (**exemplo: Whatsapp, Google Callendar, email, etc**).



A Gestão de Trade-offs na Engenharia de Software

"Arquitetura não é sobre escolher a melhor tecnologia, mas sobre escolher a tecnologia que melhor suporta o estágio atual do seu negócio."

ADRs (Architecture Decision Records)

É fundamental, que as decisões que foram tomadas em sua arquitetura e a visão por trás de tal escolha sejam registradas no decorrer do seu projeto, isso traz uma noção organizacional das vantagens que a estrutura do seu projeto trás e ajuda na tomada de decisões. As ADRs devem estar conectadas o mais próximo do seu projeto, de preferência em sua documentação.

Decisão Arquitetural	Benefício (Ganho)	Custo (Renúncia)
Integração Direta (Hard-coded)	Total controle, menor latência, custo zero de infra.	Alto acoplamento, maior esforço de dev, manutenção manual.
Integração via Middleware (n8n)	Alta flexibilidade, agilidade no desenvolvimento, low-code.	Dependência de terceiros, custo de recorrência, latência extra.
Monolito (Django)	Simplicidade, facilidade de deploy e debug, consistência de dados.	Escalabilidade limitada, risco de "código espaguete" conforme o projeto cresce.
Microserviços	Escalabilidade individual, resiliência isolada, independência tecnológica.	Complexidade operacional altíssima, latência de rede, necessidade de DevOps.

★ Exemplo prático de uma página web com Agentes de IA implementada:

🔗 https://github.com/AntonioPROLucas/Projeto_JurilA

🔗 https://youtu.be/ynSrh_bMNk8?si=dWpthV7Od4oI94Bu

OBS: (Este documento registra os fundamentos teóricos. As decisões de projeto específicas, justificativas e trade-offs de implementação (ADRs) estão versionadas no repositório GitHub, sob a pasta `/docs/adr`, garantindo que a documentação técnica evolua junto com o código-fonte.)

Multiagentes (o futuro da arquitetura com IA)

Multiagentes: Arquitetura que executa múltiplas tarefas, com um agente para cada especificidade, interagindo e compartilhando dados entre si.

Muitos agentes vs Multiagentes: Uma arquitetura é multiagente APENAS quando os agentes interagem entre si, quando os agentes estão isolados, o projeto não pode ser chamado de multiagentes.

Tipos de Arquitetura:

- **Orquestradora/Supervisora:** Como o próprio nome indica, um agente principal assume a responsabilidade por tomar decisões e delegar tarefas.

-**Vantagens:** Controle centralizado e mais fácil de supervisionar.
-**Desafios:** Se o orquestrador falhar, o sistema inteiro é comprometido.
- **Rede de Agentes:** Os agentes se comunicam de forma descentralizada, não existe hierarquia entre os agentes.

-**Vantagens:** Mais flexível e resiliente.
-**Desafios:** Mais difícil de encontrar, se existir, o agente que está com erro.
- **Hierárquica:** Os agentes são organizados em níveis. O mais altos executando tarefas de estratégia e os mais baixos executando tarefas operacionais.

-**Vantagens:** Maior escalabilidade. Mais fácil de dividir responsabilidades.
-**Desafios:** Maior latência, o fluxo passa por vários níveis.
- **Personalizada:** A mais usada em projetos. Mescla das outras arquiteturas de acordo com as necessidades de seu projeto.

Arquitetura	Cenário ideal	Trade-off Principal
Orquestradora	Fluxos lineares onde o resultado final depende de uma validação central.	Risco de gargalo no “Supervisor”.
Hierárquica	Processos complexos com várias etapas.	Maior latência.

Arquitetura	Cenário ideal	Trade-off Principal
Rede	Sistemas que exigem altíssima resiliência onde agentes trocam informações em tempo real.	Dificuldade maior em debug
Personalizada	Projetos Híbridos.	Exige documentação técnica (ADRs) rigorosa.